



SUP DE VINCI — MASTER C15

C15-TOUR

Éditeur d'itinéraires cartographiques

React 19

TS TypeScript 5.9

Vite 7

Projet privé

Application web permettant de **concevoir, calculer et partager des itinéraires touristiques** sur une carte interactive. L'organisateur place des points d'intérêt, les organise par glisser-déposer, calcule automatiquement le tracé routier, puis publie l'événement via un **code de partage** transmissible aux participants.

🕒 Projet d'étude **C15-TOUR** — Sup de Vinci (Master). Ce dépôt contient le **front-end** ; il communique avec une API back-end distante (<https://c15-tour-back.vercel.app>).

🌟 Fonctionnalités

- 📍 **Carte interactive** (Leaflet / React-Leaflet) avec recherche d'adresses et géocodage intégré.
- 📌 **Gestion de waypoints** : ajout par clic ou recherche, libellés, adresses, types de points et regroupement par **groupes colorés**.
- 🗺️ **Calcul d'itinéraire automatique** via OSRM — distance, durée et tracé complet, avec **optimisation de l'ordre** des points (problème du voyageur de commerce).
- 🖱️ **Réorganisation par glisser-déposer** des points et des groupes ([@dnd-kit](#)), panneau latéral redimensionnable desktop & mobile.
- 🔗 **Publication & partage** : chaque itinéraire publié génère un **code de partage** copiable en un clic, transmissible aux participants.
- 🔒 **Authentification** : connexion, routes protégées et **espace administrateur** (création de comptes réservée aux admins).

💡 CE QUI LE DISTINGUE

Contrairement à un simple traceur de carte, C15-TOUR modélise un **événement touristique complet** (dates, nombre de participants max, groupes, points typés) et persiste le tout côté serveur avec **sauvegarde automatique** et gestion d'état différentiel ([isDirty](#) / file d'attente de sauvegarde).

Démarrage rapide

Prérequis

- **Node.js** ≥ 20.19 (ou 22.12+) — requis par Vite 7
- **npm** ≥ 9 (ou pnpm / yarn)

Installation

```
# 1. Cloner le dépôt
git clone https://github.com/C15-TOUR-SUPDEVINCI/C15-tour-web-map-prototype.git
cd C15-tour-web-map-prototype

# 2. Installer les dépendances
npm install

# 3. Configurer l'environnement
cp .env.example .env

# 4. Lancer le serveur de développement
npm run dev
```

Ouvrez **<http://localhost:5173>** : vous serez redirigé vers la connexion, puis le tableau de bord après authentification.

Scripts disponibles

Commande	Description
<code>npm run dev</code>	Serveur de développement Vite avec HMR (port 5173)
<code>npm run build</code>	Vérifie les types (<code>tsc -b</code>) puis build de production
<code>npm run preview</code>	Sert localement le build de production
<code>npm run lint</code>	Analyse statique du code avec ESLint

Utilisation

1 — Créer un itinéraire

- 1 Depuis le **tableau de bord**, cliquez sur *Nouveau*.
- 2 Recherchez une adresse ou cliquez sur la carte pour ajouter des points.
- 3 Glissez-déposez pour réordonner, ou activez l'**optimisation** pour le trajet le plus court.

2 — Organiser par groupes

Créez des groupes colorés et répartissez les points pour structurer les étapes d'un parcours.

3 — Publier et partager

Renseignez le nom, la description, les dates et le nombre de participants max, puis publiez. Un **code de partage** est généré et copiable en un clic.

Configuration — Variables d'environnement

```
# URL de l'API back-end (obligatoire en production)
VITE_API_URL=https://c15-tour-back.vercel.app
```

Variable	Requis	Valeur par défaut	Description
VITE_API_URL	Non *	https://c15-tour-back.vercel.app	URL de base de l'API back-end

* En développement, si absente, l'app bascule sur l'URL de production avec un avertissement console.

🔌 Référence API front-end

Le client HTTP centralisé (`src/lib/apiClient.ts`) gère l'authentification par jeton Bearer et le parsing des réponses.

```
import { apiClient } from './lib/apiClient';

apiClient.get<T>(endpoint, options?) // Requête GET
apiClient.post<T>(endpoint, body?, options?) // Requête POST
apiClient.patch<T>(endpoint, body?, options?) // Requête PATCH
apiClient.delete<T>(endpoint, options?) // Requête DELETE
```

Options (`ApiClientOptions`) : `authToken?` , `skipAuth?` , plus les options natives `fetch` . Les erreurs HTTP sont levées en `ApiHttpError` avec `status` et `body` .

Service de routage — `routing.service.ts`

```
calculateRoute(waypoints: Waypoint[], options?: RouteOptions): Promise<RouteResult | null>
```

Paramètre	Type	Description
<code>waypoints</code>	<code>Waypoint[]</code>	Au moins 2 points <code>{ lat, lng }</code>
<code>options</code>	<code>RouteOptions</code>	<code>optimize?: boolean</code> , <code>algorithm?: 'fastest' 'shortest'</code>

Retour `RouteResult` : `coordinates` , `distance` (m), `duration` (s), `legs[]` , `waypointOrder?` si optimisation.

Utilitaires : `formatDistance(meters)` → `"12.40 km"` · `formatDuration(seconds)` → `"1h 30min"`

Stack technique

Domaine	Technologies
Framework	React 19, React Router 7
Langage	TypeScript 5.9
Build / Dev	Vite 7
Cartographie	Leaflet, React-Leaflet, leaflet-control-geocoder, OSRM
État global	Zustand
Drag & drop	@dnd-kit (core / sortable / utilities)
UI / Icônes	lucide-react
Qualité	ESLint, typescript-eslint

Architecture

```
src/
├─ components/  # Composants UI (Auth, Dashboard, Map, Waypoints, UI partagée)
├─ views/       # Pages routées (Login, Signup, EditorView, ...)
├─ store/       # Stores Zustand (useAuthStore, useRouteStore)
├─ services/    # Appels API & logique métier (routing, geocoding, ...)
├─ domain/      # Types & constantes du domaine
├─ lib/         # apiClient, gestion d'erreurs
└─ config.ts    # Configuration via variables d'environnement
```

👉 Contribuer

- 1 **Forkez** le dépôt et créez une branche depuis `main` :

```
git checkout -b ma-fonctionnalite
```

- 2 **Configurez** l'environnement de dev : `npm install`, puis `npm run dev`.

- 3 **Respectez le style de code** avant chaque commit :

```
npm run lint  
npm run build # vérifie aussi les types TypeScript
```

- 4 **Commits conventionnels** : `feat:`, `fix:`, `refactor:`, `docs:` ...

- 5 **Ouvrez une Pull Request** vers `main` en décrivant le contexte et les changements.

Conventions de code

- TypeScript strict — pas de `any` implicite.
- Composants fonctionnels React + hooks.
- Respect des règles `eslint-plugin-react-hooks`.
- Commentaires métier en français, cohérents avec l'existant.

Crédits

Projet **privé**, réalisé dans un cadre pédagogique (non distribué publiquement). Aucune licence open source n'est accordée — tous droits réservés aux auteurs.

Réalisé par

CHEF DE PROJET

Johnny DE OLIVEIRA

DÉVELOPPEURS

Raoul BONSSO

Nolan COHONER

Maxime KINIFFO

Coleen MICLO

Wilfried NGUEGUIM

Kyllian RONNE

Remerciements

- **OpenStreetMap** — données cartographiques libres
- **Project OSRM** — moteur de calcul d'itinéraires open-source
- **Leaflet & React-Leaflet** — bibliothèque de cartographie
- L'équipe **C15-TOUR** — Sup de Vinci